Special Participation C: Refactoring HW 11 Q4 (Scaling Laws)

Jaimyn Drake, Athul Krishnan, Mishty Dhekial

Overview

For Special Participation C, our group elected to focus on refining the style for Homework 11 Question 4: Scaling Laws. To this end, we focused on implementing the PEP8 Python style guidelines from [1] and PEP 484 type hinting [2] into the notebook for improved readability. Besides these guides, our modified version reflects generally approved Python practices, as reflected in sources like the Numpy Documentation Style Guide [4] and Google Python Style Guide [3]. Rather than refactoring the codebase into multiple files, we focused on maintaining the existing notebook structure, which we believe helps present the concepts in a digestible, linear fashion. Instead, our work focuses on the finer stylistic details that help with readability and the overall notebook experience. The revised notebooks can be found at https://github.com/Crazyturtlej/CS-C182-HW11-Refactor.

1 Method

In order to perform most of the work of refactoring, we used the built-in Gemini attachment of Google Colab, which supports the direct revision of Jupyter Notebooks within its environment. To establish our initial notebook revisions, we employed the following prompts:

1.1 Student Version (No Solutions)

Given the original assignment notebook, we prompted Gemini to revise it using the following prompt:

> "Hey Gemini! This is a deep learning code assignment, in which students explore the idea of scaling laws for hyperparameter selection. The original code was written in a rush to release to students, so now we would like to reformat the notebook to reflect best practices. In particular, we want to follow the PEP8 style guidelines for Python with PEP 484 type hinting, as well as general best practices. In revising this notebook, please make these stylistic modifications, but DO NOT FILL IN THE TODO CODE. We still want students to be able to work on the assignment themselves; just want to improve the style of the starter code that is provided."

Based on this quote alone, Gemini, was able to suggest a broad majority of the necessary line-by-line changes to the code, including:

- Spacing Improvements surrounding mathematical operations (e.g. "+", "-")

- Correct Commenting Practices (increasing the space for in-line comments to two or more spaces, as required by PEP8)
- Clarifying variable typings (e.g. including decimals after relevant hyperparameters to indicate that they are float-valued)
- Added Docstrings to Function Definitions
- Clearer Variable Names

The results of this process were impressive. However, we noticed that Gemini could sometimes be overzealous about type hinting when it comes to particular variables (e.g. specifying that the number of data samples is an integer). We believed that these addtional specifications actually served to hinder the reading experience, so we prompted Gemini to remove them.

1.1.1 Hand Tuning

With the groundwork established by Gemini, we focused on removing redundant import statements that it enjoyed adding throughout the code blocks, and modified some spacings as necessary. Because Gemini proposed its modifications as a series of line-by-line suggestions, we were able to analyze them carefully an cross-reference with the relevant documentation. After clearing the notebook's outputs, we had a cleaner version of the notebook with the exact some ordering and action items, but now in line with proper Python coding principles.

1.2 Solutions Version

In order to created a cleaned solutions version of the question, we began by copying the already-cleaned cells of the Student Version into the staff-provided solutions notebook. This way, we would ensure that both notebooks remained consistent with one another. From there, we prompted Gemini with the following prompt to apply the same appropriate changes to the question solutions:

> "Hey Gemini! This is a deep learning code assignment, in which students explore the idea of scaling laws for hyperparameter selection. The original code was written in a rush to release to students, so now we would like to reformat the notebook to reflect best practices. In particular, we want to follow the PEP8 style guidelines for Python with PEP 484 type hinting, as well as general best practices. In revising this notebook, please make stylistic modifications ONLY IN THE TODO CODE, not the rest of the existing functions which have already been revised for style. We simply want the implementation solutions to better reflect best coding practices, so that students have a better standard to learn from."

As before, this prompt allowed Gemini to parse line-by-line through the code of the solutions and apply similar improvements to those applied above. Since the solutions did not involve as much defining of new functions or utilities, there was less work to be done in terms of generating comments and doc strings.

1.2.1 Hand Tuning

With most of the changes performed by Gemini, we once again focused on removing redundant import statements that it enjoyed including throughout the code blocks (such as explicitly importin

the "List" and "Dict" types from "typings", in order to explicitly define the corresponding variables), and having it diminish unnecessary type hinting where we felt appropriate. After this, we cleared and re-ran the outputs for all of the solution notebook's cells, which helped remove some of the awkward output artifacts from the original solutions (which included a failed cell message, for example). The result of this process is a canonical solutions version that we believe is much more polished and immediately interpretable for students aiming to parse the ideas of the assignment.

2 Conclusion

With a little bit of careful prompting and observation, the Gemini support in Google Colab was incredibly easy to use and control for the purposes of refining our problem's Jupyter notebook. By focusing on maintaining the notebook's structure, we believe that our result remains easy for students to use by parsing the problem sections one-by-one, while added function descriptions, more precise definitions, and general quality improvements help augment the quality of the problem experience. We hope that this cleaned version of the assignment will be useful for future course staff, and for our future peers as a result.

References

[1] Guido van Rossum, Barry Warsaw, and Nick Coghlan. *PEP 8 – Style Guide for Python Code*. `https://peps.python.org/pep-0008/`.

[2] Guido van Rossum et al. *PEP 484 – Type Hints*. `https://peps.python.org/pep-0484/`.

[3] Google. *Google Python Style Guide*. `https://google.github.io/styleguide/pyguide.html`.

[4] Numpy developers. *NumPy Documentation Style Guide*. `https://numpydoc.readthedocs.io/en/latest/format.html`.